Concise iPRA: a smaller validated checker

Exact scope reuse, a product obstruction, and a partial uniqueness theorem

Research report prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

This report gives a modest implementation improvement to the current ConciseIPRA presentation. The full Eq/Q/D standalone checker falls from 897 to 881 normalized Python bytes, from 401 to 393 significant tokens, and from 468 to 460 AST nodes. Its exact integer/list certificate contract, public formula language, and accepted inputs are preserved. The Eq/D public variant falls from 878 to 862 bytes. Gzip-9 sizes are reported only for information. Both changes are proved to preserve acceptance and are checked against the frozen incoming implementations.

The mathematical axiomatization and its existing bi-interpretation with iPRA are unchanged. No new smaller axiom basis, global minimum, or complete new Lean verification is claimed. In particular, these validated profiles do not beat the separate 520-byte guarded profile with its different input contract.

Two mathematical investigations are recorded precisely. A closed native formula separates the existing case rule from a tempting direct-target replacement, using a coordinatewise model on $\mathbb{N}^2$. A one-premise iteration-uniqueness rule is proved to imply induction for zero equations and, more generally, for predicates admitting a term-defined retraction. The equality-reflection step needed to make that second route a complete replacement for scalar equation induction remains open in this work.

Contents

1 What is established, and what is not **3**
1.1 Matched measurements . . . 3

2 Complete inherited mathematical presentation **3**
2.1 Terms, formulas, and ordinary readings . . . 4
2.2 Every primitive axiom and rule . . . 4
2.3 Derived induction and numerical cases . . . 5
2.4 All ordinary logical operations are explicit macros . . . 5

3 Why the iPRA bi-interpretation is preserved **6**
3.1 The intermediate ordinary theory . . . 6
3.2 Exact strength of the inherited result . . . 7

4 The two implementation changes and their proofs **7**
4.1 Input contract and trusted boundary . . . 7
4.2 Sorted arity syntax . . . 7
4.3 Reuse the scope check already performed by the kernel . . . 8
4.4 Split the certificate directly . . . 9
4.5 Complete primary program . . . 10

5 Reproduction, measurements, and validation **10**

6 Why equation induction cannot replace numerical cases **11**
6.1 A product model of all the equational arithmetic . . . 11
6.2 The failure is visible inside the native language . . . 12
6.3 What remains possible . . . 12

7 A conditional route from iterator uniqueness to induction **13**
7.1 The rule and its elementary consequences . . . 13
7.2 Retractions closed under successor are the identity . . . 14
7.3 Induction from term-definable conditionals . . . 15
7.4 An unconditional application to zero equations . . . 16
7.5 The remaining bootstrap and the comparison boundary . . . 16

8 Further research questions **17**

9 Deliverable inventory and precise limits **17**

1 What is established, and what is not

The requested mathematical goal is a simpler axiomatization bi-interpretable with full first-order intuitionistic primitive recursive arithmetic. The present work establishes a smaller complete *implementation of the existing presentation*, under the proof-checker-size criterion. It does not establish a new smaller mathematical axiomatization. This distinction is essential to interpreting the result.

The repository comparison is pinned to commit

`56249a4263cae9aad9ba87d23d0ee6d4ff6483bc`.

The inspected directory is `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. Besides the main README and selected Lean sources, the comparison includes all three ZIP archives present in `docs/incoming` at this commit: `ipra_checked_refinement`, `ipra_one_sided_transport`, and `ipra_validated_core`, all dated 2026-11-11. The relevant exact controls are copied into this package; their provenance and source hashes are recorded in `PROVENANCE.json`.

This matters because the main README's compact envelope is not the newest result in every interface. The incoming checked-refinement package already has a 520-byte compact guarded checker, and the incoming validated-core package already has the 897-byte fully typed Eq/Q/D checker used here. One-sided crossed equality transport is also already present and is not claimed as a new result. These facts were read from the actual snapshot, not inferred from package names [2, 6, 7, 8].

1.1 Matched measurements

Complete program	Bytes	Tokens	AST nodes	Gzip-9
Full Eq/Q/D: incoming	897	401	468	505
Full Eq/Q/D: this package	**881**	**393**	**460**	500
Eq/D: incoming	878	390	454	498
Eq/D: this package	**862**	**382**	**446**	493

The full profile saves 16 bytes, approximately 1.78 percent. The Eq/D profile saves 16 bytes, approximately 1.82 percent. Each saves eight significant tokens and eight AST nodes. The same diagnostic text is kept; none of the saving comes from shortening an error message.

The 520-byte profile has a guarded public language and assumes that its advertised goal is canonical. The 788-byte Eq/D goal-checking profile does not enforce the same exact whole-certificate type contract. Neither is a matched comparator for the fully typed programs above. This report makes no claim that 881 or 862 is a new minimum across those different contracts. Reusing a kernel scope check already appeared in that weaker Eq/D interface; the present result composes it with the stronger whole-certificate contract.

2 Complete inherited mathematical presentation

The source transformation leaves the proof calculus unchanged. For clarity, its complete mathematical inventory is stated here. The implementation uses the source-restricted form of the inherited Choice-DI calculus. Its equivalence to the unrestricted native calculus is an inherited result, not a new theorem of this report [2, 3, 4].

2.1 Terms, formulas, and ordinary readings

There is one numerical sort, with terms

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only the update body h binds u. The counter n and initializer a are outside that binding. Arbitrary fixed numerical parameters may occur in the body. Terms are finite syntax; variables do not range over function descriptions.

Native formulas are

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A.$$

The two bound variables of D are distinct. All substitutions are capture-free, with bound variables renamed when necessary. The designated contradiction is $0 = S0$. The ordinary readings are

$$\begin{aligned}(s = t)^\circ &:= (s = t),\\ (\mathsf{Q}_y B)^\circ &:= B^\circ(0) \to \forall y\, B^\circ(y),\\ (\mathsf{D}_{x,y} B)^\circ &:= \exists x\big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big).\end{aligned}$$

These readings apply also to atomic matrices; a progressive proof instruction does not change a formula's meaning.

2.2 Every primitive axiom and rule

Contexts Γ are finite reusable lists of native formulas. The computation axioms, in any retained context, are

$$I(0, a; u.h) = a, \tag{I0}$$

$$I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{IS}$$

The other rules are exactly the following.

Hypothesis. If A is retained in Γ, then $\Gamma \vdash A$.

Equality transport. For every native formula template $F(v)$,

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}. \tag{Tr}$$

The source-restricted implementation requires that s is not the *literal syntax tree* 0. It may denote zero: for example, $I(0, 0; u.0)$ is an allowed source. All omitted literal-zero instances are derivable in this calculus. The inherited Lean kernel instead uses unrestricted transport for equation templates only and derives general formula transport; the resulting sequent relations coincide.

Closure elimination.

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}. \tag{QE}$$

Paired-binder elimination.

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x, y) \qquad \Gamma, \mathsf{Q}_y B(b, y) \vdash A}{\Gamma \vdash A}. \tag{DE}$$

Here b is fresh for Γ, A, and every fixed parameter of B; the displayed local assumption is discharged.

Introduction by cases.

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{DI-case}$$

Here z is fresh for Γ, t, and the fixed parameters of B. The matrix B may be any native formula.

Introduction by equation progress.

$$\frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y} E(x,y)}. \tag{DI-progress}$$

The same freshness condition applies, and $E(x,y)$ must be a literal equation, not an equation inside a binder. The local assumption is discharged. There are no other primitive logical, induction, successor, or equality axioms. In particular, reflexivity and explosion are derived.

The Lean presentation with direct atomic closure replaces the last rule by $\Gamma, E(z) \vdash E(Sz) \Rightarrow \Gamma \vdash \mathsf{Q}_y E(y)$. The unused-witness construction proves the equivalence of these two forms. The present Python transformation changes neither form nor any certificate instruction.

2.3 Derived induction and numerical cases

For any literal equation $E(y)$, progress introduction with an unused witness coordinate, followed by DE and QE, gives

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}. \tag{EI}$$

For an arbitrary native formula $B(y)$, the same unused-witness construction using DI-case gives

$$\frac{\Gamma \vdash B(0) \qquad \Gamma \vdash B(Sz)}{\Gamma \vdash B(t)}. \tag{Cases}$$

The second premise may ignore the retained base assumption. Freshness is as above. These two principles have different mathematical roles: Section 6 explains why EI does not replace Cases.

As a simple equality example, put $r = I(0,t;u.0)$. I0 gives $r = t$, and transport with template $v = t$, using that equation as both premises, gives $t = t$. The source is iterator-headed, so the source restriction is respected.

2.4 All ordinary logical operations are explicit macros

Let $C(v,a,b) = I(v,a;u.b)$, with u fresh. To define a formula selector without adding a connective, first define $L(v,s,t,B)$ by recursion on B:

$$\begin{aligned} L(v,s,t,r=q) &:= \big(C(v,s,r) = C(v,t,q)\big), \\ L(v,s,t,\mathsf{Q}_y B) &:= \mathsf{Q}_y L(v, C(y,0,s), C(y,0,t), B), \\ L(v,s,t,\mathsf{D}_{x,y} B) &:= \mathsf{D}_{x,y} L(v, C(y,0,s), C(y,0,t), B). \end{aligned}$$

All newly bound variables are fresh for external arguments. Define $G(v,A) = L(v,0,0,A)$. Next define Sel by recursion on its first formula argument:

$$\begin{aligned} \mathrm{Sel}(v, s=t, B) &:= L(v,s,t,B), \\ \mathrm{Sel}(v, \mathsf{Q}_y A, B) &:= \mathsf{Q}_y\, \mathrm{Sel}(v, A, G(y,B)), \\ \mathrm{Sel}(v, \mathsf{D}_{x,y} A, B) &:= \mathsf{D}_{x,y}\, \mathrm{Sel}(v, A, G(y,B)). \end{aligned}$$

The inherited selector lemmas prove that $G(0, A)$ is a theorem, $G(Sv, A)$ is equivalent to A, and $\mathrm{Sel}(0, A, B)$ and $\mathrm{Sel}(Sv, A, B)$ are equivalent to A and B, respectively.

With $\mathrm{par}(y) = I(y, 0; u.C(u, 1, 0))$, the logical macros are

$$\begin{aligned} \bot &:= 0 = S0, \\ A \to B &:= \mathsf{Q}_y \,\mathrm{Sel}(y, A, B), \\ A \wedge B &:= \mathsf{Q}_y G(y, \mathrm{Sel}(\mathrm{par}(y), A, B)), \\ \exists x A &:= \mathsf{D}_{x,y} G(y, A), \\ A \vee B &:= \exists v \ \mathrm{Sel}(v, A, B), \\ \forall x A &:= A(0) \wedge \mathsf{Q}_x A. \end{aligned}$$

These are recursive syntactic abbreviations. The existing proofs establish the full intuitionistic natural-deduction rules for them, including assumption discharge, the two quantifier eigenvariable conditions, and explosion. There is no appeal to classical excluded middle for arbitrary formulas. The complete inherited choice-basis proof is supplied in `reference/mathematics/inherited_choice_basis.tex`.

3 Why the iPRA bi-interpretation is preserved

It is useful to separate a theorem about mathematical theories from a theorem about program acceptance. The current source already establishes the former; the new proof in Section 4 establishes the latter for these modifications.

3.1 The intermediate ordinary theory

Let T_{eq} use ordinary intuitionistic first-order logic with actual equality and the same scalar terms. Its nonlogical axioms and schema are I0, IS, numerical coverage

$$\forall x \, (x = 0 \vee \exists y \, x = Sy),$$

separation $0 = S0 \to \bot$, and contextual induction EI for literal equations. The ordinary interpretations of Q and D send every native rule to a valid derivation in T_{eq}. Conversely, the macros above translate the ordinary logical rules and the nonlogical principles into native derivations. Both formula translations are faithful and their composites are provably equivalent to the original formulas.

The scalar-to-iPRA interpretation compiles each fixed scalar term into a fixed primitive-recursive description. The iterator is a primitive recursion whose update ignores the recursion counter. This is structural compilation of a finite expression, not a postulated universal primitive-recursive evaluator.

For the reverse interpretation, the inherited scalar development first constructs a predecessor satisfying $P(0) = 0$ and $P(Sn) = n$. It then constructs a faithful equality characteristic, proves quantifier-free induction by Boolean reflection, constructs pairing and projections, and compiles arbitrary fixed primitive-recursive descriptions into scalar terms. The order prevents circular reliance on pairing to obtain predecessor or on equality reflection to justify its own induction. The supplied source files `scalar_predecessor.tex`, `equality_reflection.tex`, and `primitive_recursion.tex` contain these inherited written arguments.

For each fixed primitive-recursive description, uniqueness of its recursion equations proves that translating its scalar compilation back to iPRA yields the original function. Structural induction on scalar terms, including uniqueness of each fixed iterator recurrence, proves the other term round trip. The formula statements then follow by structural induction and the native macro equivalences.

3.2 Exact strength of the inherited result

Write J for native-to-iPRA translation and K for the reverse. The source proves, for arbitrary finite contexts and open formulas,

$$\begin{aligned} \Gamma \vdash_{\mathcal{C}} A \quad &\iff \quad J\Gamma \vdash_{\text{iPRA}} J(A), \\ \Delta \vdash_{\text{iPRA}} \varphi \quad &\iff \quad K\Delta \vdash_{\mathcal{C}} K(\varphi). \end{aligned}$$

It also proves A and $KJ(A)$ interderivable and iPRA $\vdash \varphi \leftrightarrow JK(\varphi)$, with parameters. Both numerical domains remain whole and both numerical composites have identity graph $x = y$. This is bi-interpretability, with the stronger identity-domain property, rather than merely equiconsistency or agreement about standard numerical calculations.

The named Lean statements include `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff` [5]. The comparison with the reports' rule inventory is `iprf_iff_nprf` [4]. Selected exact source snapshots are included for review. They are not a standalone copy of the entire Lean project. This work inspected those sources; it did not rebuild the inherited Lean chain or formalize the Python-to-Lean correspondence.

In particular, the unchanged calculus proves $0 = S0$ exactly when iPRA proves $\bot$. The new programs accept exactly the same input strings as their matched old programs in the abstract execution sense stated below. Thus they inherit exactly the same represented theorem set and the same bi-interpretation. This last step relies on the inherited checker correctness argument; acceptance preservation does not independently re-prove that whole argument.

4 The two implementation changes and their proofs

4.1 Input contract and trusted boundary

Each selected checker is one complete Python module. It imports only the standard JSON library, reads standard input, and accepts by normal exit with output exactly `ACCEPT` followed by one newline. Any exception or other result is rejection. No axiom database, certificate producer, measurement utility, or theorem library is imported by the checker. The code does not use Python assertions for proof obligations, so `python3 -O` retains those obligations.

The input is a nonempty JSON list. The final element advertises the conclusion. Earlier elements supply four-field packets consumed from right to left. An unused prefix is allowed but is still fully checked to contain only exact integers and lists. Booleans, floats, strings, null, and dictionaries anywhere in that prefix are rejected. The full profile permits canonical Eq/Q/D conclusions; the smaller profile permits Eq/D conclusions. Internal closures remain available in both. The smaller public language still expresses the whole theory: replace $\mathsf{Q}_y B$ by $\mathsf{D}_{x,y} B$ with x absent from B, recursively throughout the formula. On the nonempty numerical domain their ordinary readings agree, and DE together with DI-case and QE proves the native interderivability. Thus the Eq/D profile does not discard the meaning of closure formulas.

4.2 Sorted arity syntax

At a formula position, arities one, two, and three mean D, equality, and Q, respectively. A public Q is represented by three identical copies of its matrix. At a numerical position, arities one and three mean successor and iterator. Canonical zero is the integer zero. The interpretation is sorted: a ternary formula and a ternary numerical term have different meanings.

The inherited encoding gives every protected numerical slot width two. At a numerical position under p protected slots, the scope depth is $e = 3 + 2p$. Canonical variable coordinates are the odd numbers $3, 5, \ldots, 2p + 1$. Under successor, two dummy zero slots are introduced; under an iterator, one state slot is introduced in its body and a dummy zero slot in its counter and initializer. These dummy slots are part of the published decoder, not extra free variables. Under D two actual numerical slots are introduced, and under Q one is introduced.

Every list edge updates the opener depth by

$$e \longmapsto e + 5 - \mathrm{len}(a).$$

The public validator uses a depth d larger by two, starting with $d = 2$ at the formula root. Thus it follows the same increments and $d = e + 2$ at every node. The complete original sorted decoder and the handling of administrative auxiliary arrays are included in `reference/mathematics/inherited_arity_checker.tex`. No new meaning for auxiliary arrays is introduced by this report.

4.3 Reuse the scope check already performed by the kernel

At a numerical leaf, the old public validator permits zero or an odd integer satisfying

$$2 < a < d - 2.$$

The new validator keeps exact integer typing, numerical position, oddness, and the lower bound, but replaces that chained comparison by $a > 2$. The omitted upper bound is enforced by the first unchanged operation of the proof kernel.

Lemma 4.1 (Initial opening enforces the omitted bound)**.** *For a public goal that passes the new shape-and-type validator, its initial no-image opening succeeds if and only if every numerical slot satisfies the old upper bound. When it succeeds, the opening returns the same raw goal.*

Proof. At the first kernel call the freshness floor is $k = 0$. The opener traverses the complete goal with a list comprehension before it checks context membership or reads an instruction. At a scalar it retains the value precisely when $0 \le a < e$. On canonical numerical positions, zero satisfies this condition. For positive allowed slots, failure is exactly $a \ge e = d - 2$, the omitted bound.

At such a failure, numerical parity makes the opener attempt to insert its default image. That default is the checking function itself, used as an invalid-image sentinel. Evaluating the opener's type-dispatch operation on a function raises `TypeError`. The checker therefore rejects before any proof packet can be consumed. No hypothesis can hide the failure, because the complete initial traversal is evaluated before membership is tested.

Conversely, when all slots satisfy the old bounds, the opener retains every leaf and recursively reconstructs exactly the same lists. Structural induction on the canonical goal proves that its output equals its input. □

Theorem 4.2 (Scope reuse preserves acceptance)**.** *Replacing only* $2 < a < d - 2$ *by* $a > 2$ *in either matched public validator preserves the successful input set.*

Proof. The raw prefix validator is unchanged, as are every arity, type, parity, lower-bound, and public-language check. A goal passing the old validator passes the new one and produces the identical first kernel state. A goal newly admitted by the relaxed validator fails the initial opening by Lemma 4.1. All later kernel operations are byte-for-byte unchanged. These observations cover both directions. □

The success-set statement abstracts from physical memory and recursion limits. The location and text of a rejection need not agree. The theorem asserts that a malformed scope cannot become a successfully checked proof, not that both implementations allocate the same amount of memory.

4.4 Split the certificate directly

The second change replaces

```
q = json.load(open(0))
a = q.pop()
```

by

```
*q, a = json.load(open(0))
```

Lemma 4.3 (Envelope preservation)**.** *The direct split preserves the successful input set under the retained validators.*

Proof. For every nonempty JSON list, it produces precisely the same prefix and last element as the old pair of statements. The checker never tests the identity of the certificate's list containers. Empty iterables and noniterable JSON roots reject. The remaining alternative JSON roots are nonempty strings and objects: unpacking either supplies a string as the alleged goal (an object is iterated through its string keys). A string cannot pass the unchanged goal-type validator. Thus no formerly rejecting root becomes accepted. □

Corollary 4.4 (Exact matched-program preservation)**.** *The delivered full checker and Eq/D checker have the same successful input sets as their respective frozen incoming controls, subject to the abstract resource qualification above. In particular, both retain exact whole-certificate integer/list validation, including unused prefixes.*

4.5 Complete primary program

The following is the entire 881-byte normalized full-language checker. It is printed as an audit aid; the executable file is authoritative.

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 != 0 else a if k <= a < d else d % 2
      and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = m == ([s, u] if s else [t] * 3) and i(t, u) == a and i(t, s) or a
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a := i(*a, t,
     2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == u) and k)], k)

def v(a, d=0):
    return not d or (d % 2 and (not a or (a > 2 and a % 2))) if type(a) is int else a * 0 == [] and (not d or
     len(a) in (1, 2 + d % 2) or a == [a[0]] * 3) and all((v(b, d and d + 5 - len(a)) for b in a))
*q, a = json.load(open(0))
if not (v(q) and v(a, 2)):
    raise ValueError('The certificate must use integer/list syntax and have a canonical closed goal')
q = q.pop
c(a)
print('ACCEPT')
```

The readable standalone implementation, `code/checker_readable.py`, uses descriptive names and comments. It is supplied for review and was included in the differential process checks. Its larger source size is not substituted for the counted program.

5 Reproduction, measurements, and validation

From the extracted package root, run

```
python3 code/checker_full.py < examples/computation.json
python3 -O code/checker_full.py < examples/computation.json
python3 reproduce.py
python3 build_report.py
```

The first two commands print `ACCEPT`. The reproduction command is offline and uses the Python standard library. The document build requires `pdflatex` and the standard LaTeX packages listed in this source.

The frozen measurement runtime was CPython 3.12.14. Normalize a whole module by `ast.unparse(ast.parse(source))` plus a final linefeed. Bytes mean the UTF-8 size of that normalized source. Significant tokens exclude comments, encoding/end markers, indentation, dedentation, and newline tokens. AST nodes count every node yielded by `ast.walk`. Gzip is level nine with `mtime=0`, including the gzip wrapper. Its size is informational, not an optimization target. The normalized gzip artifacts and exact source SHA-256 values are saved in `evidence/`.

The packaged verification was actually executed. It recorded 13,019 scope-preflight comparisons *per interface*, with 3,096 valid full-language examples and 2,233 valid Eq/D examples, and no discrepancy. It also ran 606 complete checker processes across the old and new programs, the readable full implementation, and ordinary and optimized Python, with no unexpected outcome. Thirty additional targeted controls cover non-list JSON roots, high positive slots, negative scope escapes, Boolean/float/null aliases, malformed unused prefixes, and valid inert integer/list prefixes.

The preflight checks exercise the actual first kernel opening with a sentinel instruction provider; reaching the proof boundary is distinguished from accepting a proof. The process checks exercise the complete input-to-output programs. The detailed records are in `evidence/validation.json`; the structural proofs above supply the general preservation argument that finite testing cannot supply.

6 Why equation induction cannot replace numerical cases

A tempting simplification changes the case introduction from a proof of $B(t, Sz)$ under $B(t, 0)$ to a proof of $B(t, z)$ under $B(t, 0)$, with z fresh. The modified rule is sound for the ordinary conditional-closure reading without any numerical coverage axiom. It is nevertheless strictly weaker when combined with the remaining native rules and literal-equation induction. This section gives both a semantic obstruction and a closed native formula separating the two calculi. These results do not assert that every possible alternative interpretation of the weaker theory is impossible.

6.1 A product model of all the equational arithmetic

Let $\mathcal{A}$ have numerical domain $\mathbb{N} \times \mathbb{N}$. Interpret zero, successor, and iteration coordinatewise:

$$0^{\mathcal{A}} = (0, 0), \qquad S^{\mathcal{A}}(a, b) = (a + 1, b + 1).$$

For each finite iterator body, the first output coordinate is the usual natural-number iteration of its first-coordinate interpretation, and the second coordinate is defined in the same way. Equality is actual equality of ordered pairs. Use ordinary two-valued first-order semantics; a classical structure is also a sound test of intuitionistic derivability.

More explicitly, if ρ assigns pairs to the free variables, write ρ_i for its ith-coordinate assignment. Structural induction on a scalar term t gives

$$[\![t]\!]_{\mathcal{A},\rho} = ([\![t]\!]_{\mathbb{N},\rho_1}, [\![t]\!]_{\mathbb{N},\rho_2}). \tag{1}$$

The iterator case follows by the defining coordinatewise interpretation, including the coordinatewise assignment to its bound state variable. Consequently, for every literal equation $E(n, \vec{p})$ and every fixed parameter assignment, there are unary predicates E_1, E_2 on the standard natural numbers such that

$$\mathcal{A} \models E((a, b), \vec{p}) \quad \Longleftrightarrow \quad E_1(a) \text{ and } E_2(b).$$

This assertion concerns equations. Arbitrary formulas containing implication or quantifiers need not factor this way.

Both iterator computation equations hold in $\mathcal{A}$, successor is injective, and $0 \neq S0$. Moreover, every literal-equation induction instance is valid. To prove the latter claim, suppose its base and step hold. The base supplies $E_1(0)$ and $E_2(0)$. Given $E_1(a)$, evaluate the product step at $(a, 0)$, using $E_2(0)$, to obtain $E_1(a + 1)$. Similarly, evaluating the step at $(0, b)$ gives $E_2(b) \Rightarrow E_2(b + 1)$. Ordinary induction in each standard coordinate yields $E_1(a)$ and $E_2(b)$ for all a, b. This proves the product induction conclusion.

The argument also proves soundness of contextual equation induction. Fix an assignment making every retained assumption in Γ true. Freshness of the induction variable means that changing its pair value does not change the truth of Γ. The preceding argument therefore applies to the base and step under that same assignment. No restriction to equational side assumptions is required.

Numerical coverage fails. The element $(0, 1)$ is neither $(0, 0)$ nor a successor: every successor has two positive coordinates. Thus

$$\mathcal{A} \not\models \forall x (x = 0 \vee \exists y\, x = Sy). \tag{2}$$

Adding all valid predecessor computation equations does not remove this obstruction: coordinatewise predecessor already satisfies $P(0) = 0$ and $P(Sx) = x$ in this model.

6.2 The failure is visible inside the native language

Interpret the native binders by their ordinary readings

$$Q_y B := B(0) \to \forall y\, B(y), \qquad D_{x,y} B := \exists x\big(B(x,0) \to \forall y\, B(x,y)\big).$$

Hypothesis, equality transport, QE, and DE are sound in $\mathcal{A}$. The two computations and equation progress are sound by the preceding argument. The proposed direct-target DI rule is sound as well: its fresh variable already ranges over the whole domain. Hence every theorem of that modified calculus is valid in $\mathcal{A}$.

Write $1 = S0$ and $C(v,a,b) = I(v,a;u.b)$, with u absent from b. Define the following native formulas, where the displayed E is a single literal equation:

$$\begin{aligned} E(n,y,p): &\quad C\big(p, C(n,y,1), C(n,1,0)\big) = 0, \\ P(n,p) &:= Q_y E(n,y,p), \\ G(p) &:= Q_n P(n,p), \\ F &:= Q_p G(p). \end{aligned}$$

The last formula is closed.

The original calculus proves F without equation induction. At a successor $n = Sz$, computation makes $E(Sz,y,p)$ equivalent to $C(p,1,0) = 0$, independently of y. Thus $P(Sz,p)$ follows by conditional-closure introduction from that same equation as a retained assumption. The original case introduction, with an unused witness, therefore proves $Q_n P(n,p)$: its successor premise is already proved and its retained zero premise can be ignored. A further such introduction, now in the parameter p, proves F. Only case DI, DE, QE, computations, and equality reasoning are needed for these derived closure introductions.

In $\mathcal{A}$, take $p_* = (0,1)$. Direct coordinatewise computation gives

$$\mathcal{A} \models E(n,y,p_*) \quad \iff \quad n_1 = 0, \quad n_2 > 0, \quad y_1 = 0.$$

Consequently $P(0,p_*)$ is true, because its antecedent is false. Likewise $P(Sz,p_*)$ is true for every z. But $P((0,1),p_*)$ is false: its antecedent holds at $y = 0$, whereas its universally quantified consequent fails at $y = (1,0)$. Thus $G(p_*)$ is false.

Finally, $G(0)$ is true. At $p = 0$ and $n = 0$, the equation E becomes $y = 0$, so $P(0,0) = Q_y(y = 0)$ is false in this nontrivial structure. That false antecedent makes $G(0)$ true. Since $G(p_*)$ is false, the closed formula $F = Q_p G$ is false. Soundness of the modified calculus in $\mathcal{A}$ therefore proves that it cannot derive F, while the original calculus does derive F.

6.3 What remains possible

The obstruction distinguishes algebraic Boolean identities from global logical alternatives. For example, equation induction proves $t(C(n,a,b)) = C(n,t(a),t(b))$ for each fixed term context t. It can also prove that the Boolean selector value $b = C(n,0,1)$ satisfies $C(b,0,1) = b$. Such identities remain valid for the mixed Boolean pairs $(0,1)$ and $(1,0)$. They do not entail $b = 0 \vee b = 1$. Accordingly, improving the equational predecessor construction cannot by itself recover the missing numerical case rule.

There is a separate, unfinished route through recursion uniqueness. Much of the scalar predecessor bootstrap can be recast as uniqueness of an explicitly displayed recurrence. Write $Z(u) = C(u, 1, 0)$, $N(n) = I(n, 0; u.Z(u))$, and $a + b = I(b, a; u.Su)$, with $2a = a + a$. For example, if $j(u) = Z(u) + 2N(u)$, $M(n) = I(n, 0; j)$, and $G(n) = I(n, 0; u.u + M(u) + 1)$, the difficult comparison $M(G(n)) = N(n)$ can compare two recurrences with the common update $k(v) = I(Sv, v; j)$. Finite Boolean computations give $k(N(n)) = Z(N(n))$. This is useful progress on that route, but it does not complete equality reflection or establish equivalence of a new uniqueness-only theory with iPRA; no such conclusion is claimed here.

7 A conditional route from iterator uniqueness to induction

This section records an arithmetic research direction, not a completed replacement for the delivered checker. Its positive result is a precise retraction argument: scalar iterator uniqueness implies induction for every predicate for which a suitable numerical conditional is already available. In particular, it implies induction for equations of the form $t(n) = 0$ using only numerical coverage and the ordinary zero test. The missing step is an independent equality-reflection bootstrap for the scalar language. No new bi-interpretation or smaller complete checker is claimed for the proposed uniqueness rule.

7.1 The rule and its elementary consequences

Uniqueness of recursion as an alternative arithmetic principle is classical: Tait states the equivalence for a richer language with counter-aware primitive recursion and product types [1]. That result alone does not settle the restricted scalar update $h(u)$ considered here. No novelty is claimed for the general idea of using recursion uniqueness.

Work, for this arithmetic analysis, in ordinary intuitionistic first-order logic with actual equality. The numerical terms are

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h),$$

with capture-free substitution; u is bound only in h. Retain the two computation equations

$$I(0, a; u.h) = a,$$
$$I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

Terms such as $f(z)$ and $h(u)$ below are fixed finite term templates, not variables of a function sort. Passive parameters are suppressed.

Consider replacing literal-equation induction by the rule

$$\frac{\Gamma \vdash f(Sz) = h(f(z))}{\Gamma \vdash f(t) = I(t, f(0); u.h(u))}. \tag{UR}$$

Here z is fresh for Γ, the target term t, and all passive parameters; the update template $h(u)$ does not contain z. Its only active argument is the previous state u. The displayed premise is proved in Γ itself: there is no discharged induction hypothesis. The seed $f(0)$ is determined by f, so no separate base premise or seed certificate occurs in this rule.

Equation induction proves UR. Assume the premise of UR and induct on the literal equation

$$f(n) = I(n, f(0); u.h(u)).$$

The base is the zero computation equation and symmetry. At a successor, the recurrence premise, the current induction equation, congruence, and the successor computation equation give

$$\begin{aligned} f(Sn) &= h(f(n)) \\ &= h(I(n, f(0); u.h(u))) \\ &= I(Sn, f(0); u.h(u)). \end{aligned}$$

Thus UR is sound in the old arithmetic. This direction needs neither coverage nor a numerical equality test.

Uniqueness with a common initial value. UR implies the following useful derived rule. If, with fresh z,

$$\Gamma \vdash f(0) = g(0), \qquad \Gamma \vdash f(Sz) = h(f(z)), \qquad \Gamma \vdash g(Sz) = h(g(z)),$$

then $\Gamma \vdash f(t) = g(t)$. Apply UR to f and g separately and use equality congruence in the seed of the iterator. The argument is contextual: arbitrary retained first-order assumptions may occur in Γ, provided they do not contain the eigenvariable.

The successor iterator is the identity. Taking $f(z) = z$ and $h(u) = Su$, the premise of UR is reflexivity. Consequently

$$\Gamma \vdash I(t, 0; u.Su) = t. \tag{Id}$$

This identity is a consequence of UR, not an additional axiom. In particular, the retraction argument below does not silently assume that a successor orbit already exhausts the numerical domain.

7.2 Retractions closed under successor are the identity

Retraction lemma. Let $e(x)$ be a fixed scalar term template. Suppose that Γ proves the following equations, with x fresh:

$$e(0) = 0, \tag{3}$$
$$e(e(x)) = e(x), \tag{4}$$
$$e(S(e(x))) = S(e(x)). \tag{5}$$

Then the computation equations, equality reasoning, and UR prove $\Gamma \vdash e(t) = t$ for every target term t.

Proof. Define two finite term abbreviations

$$h(u) = e(S(e(u))), \qquad r(n) = I(n, 0; u.h(u)).$$

No new operation or function quantifier is introduced. Idempotence implies the two global commutation equations

$$e(h(u)) = h(u), \tag{6}$$
$$h(e(u)) = h(u). \tag{7}$$

For the first, substitute $S(e(u))$ for x in (4). For the second, unfold h and replace $e(e(u))$ by $e(u)$ under S and e. The successor-closure assumption is exactly

$$h(u) = S(e(u)). \tag{H}$$

Now consider $f(n) = e(r(n))$. Its base is $f(0) = e(0) = 0$. Its successor equation is, using the iterator computation and (6)–(7),

$$f(Sn) = e(h(r(n))) = h(r(n)) = h(e(r(n))) = h(f(n)).$$

UR, followed by replacement of the equal seeds, therefore yields

$$e(r(n)) = I(n, 0; u.h(u)) = r(n). \tag{R1}$$

This is the first substantive use of uniqueness; it establishes that the entire h orbit lies in the image of the retraction.

Equation (H) and (R1) now give

$$r(Sn) = h(r(n)) = S(e(r(n))) = S(r(n)), \qquad r(0) = 0.$$

Apply UR again, this time to r with update $u \mapsto Su$. By (Id),

$$r(n) = I(n, 0; u.Su) = n. \tag{R2}$$

Combining (R1) and (R2) gives $e(n) = n$. Every displayed calculation is an equality derivation under the fixed context. No induction is used in the proof. □

7.3 Induction from term-definable conditionals

The preceding lemma gives a useful sufficient condition for induction. Let $E(n)$ be a fixed formula. Suppose that the retained context proves its base and that its step is derivable:

$$\Gamma \vdash E(0), \qquad \Gamma, E(z) \vdash E(Sz), \tag{P}$$

where z is fresh. Suppose also that E is decidable in this context and has a term-defined split: there is a fixed term $e(n)$ such that, uniformly in fresh n,

$$\Gamma \vdash E(n) \vee \neg E(n), \tag{8}$$

$$\Gamma, E(n) \vdash e(n) = n, \tag{9}$$

$$\Gamma, \neg E(n) \vdash e(n) = 0. \tag{10}$$

Then UR proves the induction conclusion $\Gamma \vdash E(t)$.

Here is the complete reduction. A case split using (8) proves $E(e(n))$: in the $E(n)$ branch use (9); in the other branch use (10) and the base $E(0)$. The true-branch identity, instantiated at $e(n)$, gives

$$e(e(n)) = e(n).$$

The step from (P), instantiated at $e(n)$, gives $E(S(e(n)))$. Another use of the true-branch identity yields

$$e(S(e(n))) = S(e(n)).$$

Finally, the base and the true-branch identity at zero yield $e(0) = 0$. Thus all three premises of the retraction lemma hold. It follows that $e(n) = n$, and transporting the already proved $E(e(n))$ along this equality gives $E(n)$, then $E(t)$.

This result applies only to predicates whose displayed numerical split has been justified. It does not supply induction for arbitrary quantified formulas. In particular, no general principle of deciding first-order formulas or converting them to numerical tests is being assumed.

7.4 An unconditional application to zero equations

For this subsection also assume numerical coverage and separation:

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad 0 = S0 \to \bot.$$

Define the selector, as in the inherited scalar language, by

$$C(v, a, b) = I(v, a; u.b).$$

Its two endpoint equations are immediate computation:

$$C(0, a, b) = a, \qquad C(Sv, a, b) = b.$$

Coverage supplies the corresponding case distinction for every argument. Moreover, $Sx = 0$ is contradictory: applying the term $v \mapsto C(v, 0, S0)$ gives $S0 = 0$, and separation then gives $\bot$. Hence every zero equation $q(n) = 0$ is decidable, without constructing a test for equality of two arbitrary terms.

Take $E(n)$ to be the literal equation $q(n) = 0$ and set

$$e(n) = C(q(n), n, 0).$$

If $E(n)$ holds, the zero selector equation gives $e(n) = n$. If $\neg E(n)$ holds, coverage for $q(n)$ eliminates its zero alternative; in its successor alternative the other selector equation gives $e(n) = 0$. Thus (8)–(10) have direct proofs. The conditional induction result proves the following derived rule:

$$\frac{\Gamma \vdash q(0) = 0 \qquad \Gamma, q(z) = 0 \vdash q(Sz) = 0}{\Gamma \vdash q(t) = 0}. \tag{ZI}$$

Only coverage, separation, selector computation, equality reasoning, and UR were needed. No predecessor, pairing, multiplication, general primitive-recursion compiler, or faithful equality characteristic was used.

The distinction between zero equations and arbitrary equations is essential. Coverage decides whether a single number is zero. It does not itself give a term $\delta(a, b)$ satisfying $\delta(a, b) = 0 \leftrightarrow a = b$ for arbitrary a, b.

7.5 The remaining bootstrap and the comparison boundary

If the UR-based arithmetic could independently construct a scalar term $\delta(a, b)$ and prove

$$\delta(a, b) = 0 \quad \longleftrightarrow \quad a = b \tag{EqRef}$$

for actual equality throughout its domain, then ZI would recover literal-equation induction. Given an induction on $s(n) = t(n)$, apply ZI to $\delta(s(n), t(n)) = 0$ and transport its base, step, and conclusion through (EqRef). Together with the earlier proof that equation induction implies UR, this would establish equivalence of the two arithmetic principles over the stated common background.

The existing scalar proof of (EqRef) cannot simply be imported at this point. Its predecessor and reflection developments use literal-equation inductions with nonconstant right sides, including reconstruction invariants. Using those results to justify a replacement for that very induction would be circular unless their proofs were translated to UR or ZI. The argument above removes part of the obstacle by deriving ZI, but does not complete that translation.

There is a separate logical bookkeeping issue for a proposed native Q, D kernel. The proofs in this section use ordinary intuitionistic equality logic, contextual case distinction, substitution, and cut. When replacing a primitive rule in the native kernel, the required native versions of these resources must be derived without appealing to the removed rule. The ordinary arithmetic lemma alone is not a certificate of equivalence between the complete native calculi.

Finally, a premise with no local induction hypothesis need not produce a smaller implementation. UR replaces the hypothesis extension by construction of $f(t)$, $f(0)$, and an iterator with update h. Its complete syntax checks, eigenvariable checks, logical rules, and public theorem interface would all have to be measured. No such complete smaller checker or completed bi-interpretation is asserted here. The precise further research problem is to finish the equality-reflection bootstrap under UR, then perform a matched implementation comparison of the resulting complete calculus.

8 Further research questions

The following questions are deliberately separated from established claims.

1. Can equality reflection for arbitrary scalar equations be proved from scalar iteration uniqueness, coverage, separation, and the two computation laws, without invoking the equation-induction rule being replaced? The retraction theorem reduces the final induction step to that concrete bootstrap problem.

2. Can the predecessor development be completed under uniqueness with a fully explicit list of common recurrences, then extended through the residual equality-reflection invariant? This requires checking each dependency rather than importing a classical equivalence theorem for a different primitive-recursive language.

3. If the uniqueness route succeeds mathematically, does its complete standalone checker improve the current size coordinates? Removing an induction hypothesis alone does not answer this: its conclusion constructs several term instances and an iterator.

4. Can a restriction of numerical cases avoid the product countermodel while still deriving all native formula cases, with less checking machinery? A successful proposal must explain where global logical case distinction enters; equational predecessor identities alone do not do so.

5. Can the full typed arity checker be connected to the existing Lean native semantics by a proved decoder and certificate-checking theorem? That would strengthen the verification status of the whole family beyond the written preservation proofs and regression checks supplied here.

9 Deliverable inventory and precise limits

The archive contains this compiled report and its complete LaTeX source, the three standalone checker implementations, exact matched controls, inherited test certificates, the preservation proof, reproducible metrics and tests, deterministic gzip-9 artifacts, selected Lean source snapshots, the inherited scalar-to-iPRA written proofs, a provenance ledger, and a SHA-256 manifest.

The new verified result is the 881/862-byte implementation reduction with the stated matching contracts. The mathematical rule inventory in Section 2 is inherited. The product obstruction is a proved rejection of a proposed weakening; the uniqueness results are proved conditional progress toward another presentation. Neither is represented as a completed simpler iPRA axiomatization. Absolute minimality and a new formal proof of Python correctness remain unclaimed.

References

[1] W. W. Tait, *Primitive Recursive Arithmetic and its Role in the Foundations of Arithmetic: Historical and Philosophical Reflections*, author manuscript, Section 2 (pp. 2–3). https://home.uchicago.edu/~wwtx/PRA2.pdf.

[2] Vladimir Reshetnikov, *Concise presentations of intuitionistic primitive recursive arithmetic*, ProveIt README, pinned commit `56249a4263cae9aad9ba87d23d0ee6d4ff6483bc`. https://github.com/VladimirReshetnikov/ProveIt/blob/56249a4263cae9aad9ba87d23d0ee6d4ff6483bc/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/README.md.

[3] ProveIt, *Native.lean*, same snapshot. https://github.com/VladimirReshetnikov/ProveIt/blob/56249a4263cae9aad9ba87d23d0ee6d4ff6483bc/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/Native.lean.

[4] ProveIt, *NativeInherited.lean*, same snapshot; theorem `iprf_iff_nprf`. See also the README's `zprf_iff_nprf` comparison and the written source-restriction proofs in the inherited packages. https://github.com/VladimirReshetnikov/ProveIt/blob/56249a4263cae9aad9ba87d23d0ee6d4ff6483bc/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/NativeInherited.lean.

[5] ProveIt, *NativeBiInterpretation.lean* and *BiInterpretation.lean*, same snapshot. https://github.com/VladimirReshetnikov/ProveIt/blob/56249a4263cae9aad9ba87d23d0ee6d4ff6483bc/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/Lean/ConciseIPRA/NativeBiInterpretation.lean.

[6] ProveIt incoming archive `ipra_validated_core_2026-11-11.zip`, Git blob `63b42195dc702cf115fbb9f472e96355427c9e95`. https://github.com/VladimirReshetnikov/ProveIt/blob/56249a4263cae9aad9ba87d23d0ee6d4ff6483bc/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/docs/incoming/ipra_validated_core_2026-11-11.zip.

[7] ProveIt incoming archive `ipra_checked_refinement_2026-11-11.zip`, Git blob `0583b5629b493c316d01b85d410961bf3e32866b`. https://github.com/VladimirReshetnikov/ProveIt/blob/56249a4263cae9aad9ba87d23d0ee6d4ff6483bc/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/docs/incoming/ipra_checked_refinement_2026-11-11.zip.

[8] ProveIt incoming archive `ipra_one_sided_transport_2026-11-11.zip`, Git blob `b3633919ebdc8ba0f66f4f522af9b595dc583bff`. https://github.com/VladimirReshetnikov/ProveIt/blob/56249a4263cae9aad9ba87d23d0ee6d4ff6483bc/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/docs/incoming/ipra_one_sided_transport_2026-11-11.zip.